Who Wrote It? Identifying LLMs from Their Responses

CMPSC 448 Midterm Project · October 11, 2026

Team leader: Alvin James · **Team members:** Aryan Patel, Anurag Palakurti, Somay Bansal

GitHub Repo: https://github.com/alvinj024/llm-fingerprint

Overview

The goal of this project is to determine whether different language models have recognizable writing habits. We built a classifier that predicts which of four families (GPT, Claude, Gemini, or Llama) produced a response. We trained the required CNN and RNN (a bidirectional LSTM), compared them with simple baselines, and investigated all four research questions. On the test set, the output-only CNN reached 74.4 macro-F1 and the BiLSTM reached 67.8, while chance accuracy is 25%. The user's prompt was much less useful. We also tested how the models handle unseen task types and examined which words and formatting choices helped identify each family.

1. Problem Definition and Dataset Curation

1.1 Problem definition

Each data point in this project is a triple of (LLM_name, LLM_input, LLM_output), which is the model family, the user's prompt, and the model's answer. We compared three prediction settings: using only the response, using only the prompt, and using both. The main score is macro-F1 because it gives each of the four families the same importance. Accuracy is also reported. Because the dataset is balanced, chance accuracy is 25%. A classifier that always picks one family would still receive 25% accuracy but only 10% macro-F1.

1.2 Data source

The data comes from the public LMSYS Chatbot Arena human-preference-55k dataset on Hugging Face, which has roughly 55,000 rows [1, 2]. In each Arena battle, two randomly assigned models answer the same real user prompt, so each battle became two (family, prompt, response) examples. One strength of this dataset is that the prompts came from actual users and were not written separately for each model family. This matters for RQ2, which asks whether the prompt itself gives away the model. Only single-turn examples with nonempty prompts and responses were kept. The downloading and filtering code is in scripts/download_and_filter.py, and no raw user prompts are included in this report.

1.3 Mapping model names to families

Model versions were grouped into families using fixed rules. GPT-3.5 and GPT-4 versions became GPT, claude-* versions became Claude, gemini-* versions became Gemini, and official Llama-2 chat models became Llama. Other model families, third-party Llama fine-tunes (including Vicuna and llama2-70b-steerlm-chat), base LLaMA, and CodeLlama were excluded. This keeps the Llama group focused on the official chat models instead of a mix of unrelated fine-tunes. All versions are older models from 2023

through January 2024. GPT includes 8 versions (GPT-3.5 and GPT-4), Claude has 4 versions, Gemini has 2 versions (gemini-pro and gemini-pro-dev-api), and Llama includes the 7B, 13B, and 70B Llama-2 chat models.

1.4 Cleaning and balancing

Step	Examples
Single-turn examples in the four families	50,604
After English filter (prompt and response at least 90% ASCII letters)	49,631
After removing exact duplicates of (family, prompt, response)	49,186
Random sample of 1,000 per family	4,000

The families were not balanced before sampling. GPT had 26,455 examples, Claude had 13,632, Llama had 6,689, and Gemini had 2,410. Therefore, 1,000 examples were randomly chosen from each family for a total of 4,000. This kept the classes equal and made repeated experiments practical on a CPU. The unused examples were later used for the collection-bias experiment in Section 4.1.

1.5 Train, validation and test split

The data was split into 70% training, 15% validation, and 15% testing, which is 700, 150, and 150 examples per family. This gives 2,800 training examples and 600 each for validation and testing. An important detail is that the split was grouped by normalized prompt. The Arena sometimes repeats prompts, and the two responses in a battle share the same prompt. Without grouping, the same prompt could appear in both training and testing, which would make the prompt-only results misleading. The code checks that this does not happen. There were 3,822 distinct prompts in the 4,000-example sample.

```
PS C:\Users\alvin\Downloads\llm-fingerprint\llm-fingerprint> python -m src.data
raw single-turn examples in 4 families          50,604
after English filter                            49,631
after removing exact duplicates                 49,186
after balancing (1000 per family)                4,000
split   test  train  val
family
GPT      150    700  150
Claude   150    700  150
Gemini   150    700  150
Llama    150    700  150
```

For RQ3, prompts were labeled as coding, math, writing, or general using keyword rules in src/textproc.py. The counts were 501 coding, 168 math, 433 writing, and 2,898 general. The rules were tested on separate samples of 30 prompts per label, which gave estimated precision of 87% for coding, 80% for writing, 80% for general, and 50% for math. Because the math labels were especially noisy, math was not used as a held-out test category in RQ3.

2. CNN and RNN Implementation and Training

Both required neural models were implemented in PyTorch in src/models.py and src/train.py. They used the same training procedure so the comparison would be reasonably fair, and two simpler baselines were also run.

2.1 Text representation

A word-level tokenizer was written that keeps capitalization, punctuation, line breaks, and Markdown characters such as **, #, bullets, and code fences. These details were kept because formatting might be part of a model's writing style, which is examined in RQ4. The vocabulary was built from training data only, with a minimum frequency of 2 and a maximum of 30,000 tokens. It contained 21,365 tokens for output-only and 24,003 for input-plus-output. Responses were limited to 512 tokens and prompts to 256, which truncated 9.4% of responses and 6.0% of prompts. The median lengths were 246 response tokens and 17 prompt tokens.

2.2 Architectures

	CNN	**RNN (BiLSTM)**
Embedding	128-d, learned from scratch	128-d, learned from scratch, dropout 0.3
Encoder	Parallel 1-D convolutions, widths 3, 4, 5, with 128 filters each, ReLU	1 layer, 128 units per direction
Pooling	Max over time, padding masked out	Additive attention (masked softmax) over time steps
Output head	Dropout 0.5, linear layer to 4 classes	Dropout 0.5, linear layer to 4 classes
Parameters (output only)	2.93 M (2.73 M are embeddings)	3.02 M
Parameters (input + output)	6.54 M	6.71 M

CNN. We used a TextCNN [3] with three sets of 1-D filters that cover windows of 3, 4, and 5 tokens, with 128 filters in each set. A filter can learn a short pattern, such as a bullet marker followed by particular words. Max pooling keeps the strongest match across the response, so the CNN does not need to know exactly where the pattern appeared. We expected this to work well for repeated formatting and phrasing habits.

RNN (BiLSTM). The recurrent model uses one LSTM in each direction with 128 hidden units per direction, which lets it read the context around each token. Additive attention then combines the information across positions, and padding is masked so empty positions do not affect the result. We chose an LSTM instead of a basic RNN because many responses are hundreds of tokens long.

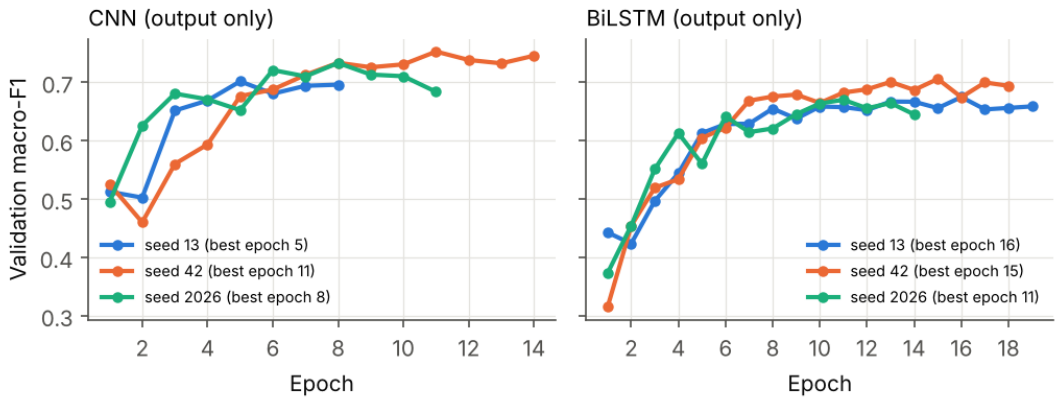
Input plus output. For this RQ2 setting, the prompt and the response are encoded separately by the same type of network, and the two encoded vectors are joined before the final classifier.

2.3 Baselines

A majority-class predictor was used as a basic reference point. The stronger baseline was TF-IDF with word unigrams and bigrams (about 75,000 features) followed by logistic regression. The regularization parameter C was chosen using the validation set, searching values from 0.01 to 1,000.

2.4 Training procedure

Both neural models used cross-entropy loss and AdamW with a learning rate of 0.001 and weight decay of 0.0001. We used a batch size of 64, gradient clipping at 1.0, and a maximum of 30 epochs. Training stopped if validation macro-F1 did not improve for three epochs, and the best checkpoint was restored. Seeds 13, 42, and 2026 were run, and the mean and standard deviation are reported. On our two-core CPU, a CNN run took around 1 to 2 minutes and a BiLSTM run around 4 to 5 minutes. The CNN's best epochs were 5, 8, and 11, and the BiLSTM's were 16, 11, and 15.




```
PS C:\Users\alvin\Downloads\llm-fingerprint\llm-fingerprint> python -m src.train --model cnn --view output --seed 13
  ep  1 loss 1.6600 val_f1 0.5124 val_acc 0.5517 (8s)
  ep  2 loss 1.2296 val_f1 0.5005 val_acc 0.5667 (16s)
  ep  3 loss 1.0499 val_f1 0.6477 val_acc 0.6417 (24s)
  ep  4 loss 0.8641 val_f1 0.6705 val_acc 0.6733 (31s)
  ep  5 loss 0.7732 val_f1 0.6986 val_acc 0.6900 (37s)
  ep  6 loss 0.6991 val_f1 0.6878 val_acc 0.6783 (44s)
  ep  7 loss 0.6376 val_f1 0.6796 val_acc 0.6700 (52s)
  ep  8 loss 0.6068 val_f1 0.6932 val_acc 0.6917 (59s)
TEST cnn/output/seed13: acc 0.7033 macro-F1 0.7106  (best epoch 5, 59s)
```

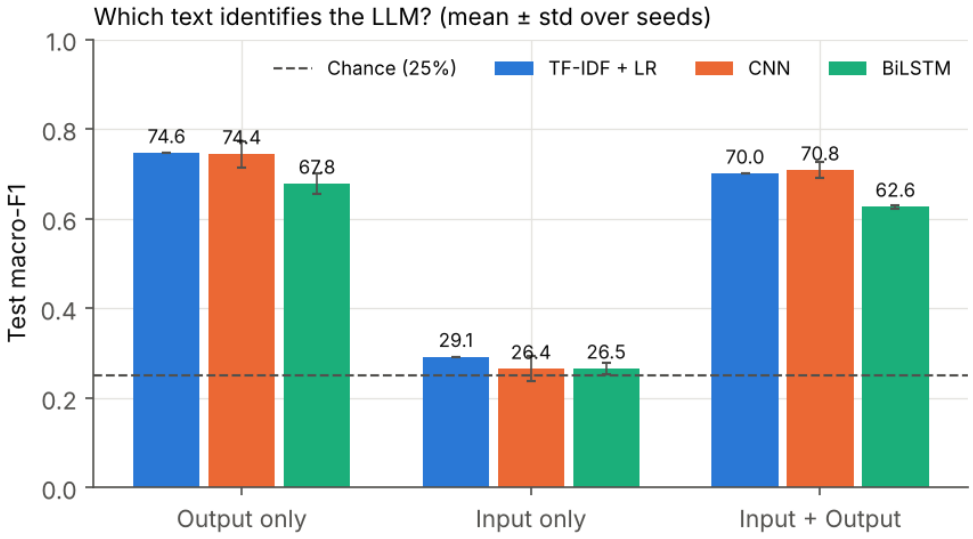
2.5 Implementation notes

- **BiLSTM speed problem.** Our first packed-sequence BiLSTM implementation was much slower than expected on CPU, at about 293 seconds per epoch. We replaced it with two one-directional LSTMs, where the backward one reads each sequence in reverse within its actual length. A test (tests/test_bilstm_equivalence.py) confirmed that it gives the same outputs as PyTorch's packed BiLSTM, and the epoch time dropped to about 17 seconds.
- **Training adjustments.** We first capped training at 15 epochs, but the BiLSTM was still improving, so the limit was increased to 30. We also widened the logistic-regression C search because some earlier runs kept selecting the edge of the grid.

3. Results: RQ1 and RQ2

This table gives the test-set results on 600 examples. For the neural models, accuracy and macro-F1 are the mean and standard deviation across three seeds.

Model	Setting	Accuracy	Macro-F1
Majority	n/a	25.0	10.0
TF-IDF + LR	Output only	74.7	74.6
CNN	Output only	74.1 ± 3.2	74.4 ± 2.9
RNN (BiLSTM)	Output only	67.7 ± 2.2	67.8 ± 2.2
TF-IDF + LR	Input only	29.2	29.1
CNN	Input only	27.4 ± 1.4	26.4 ± 2.8
RNN (BiLSTM)	Input only	26.8 ± 1.3	26.5 ± 1.4
TF-IDF + LR	Input + output	70.3	70.0
CNN	Input + output	70.8 ± 1.2	70.8 ± 1.8
RNN (BiLSTM)	Input + output	62.4 ± 0.7	62.6 ± 0.4

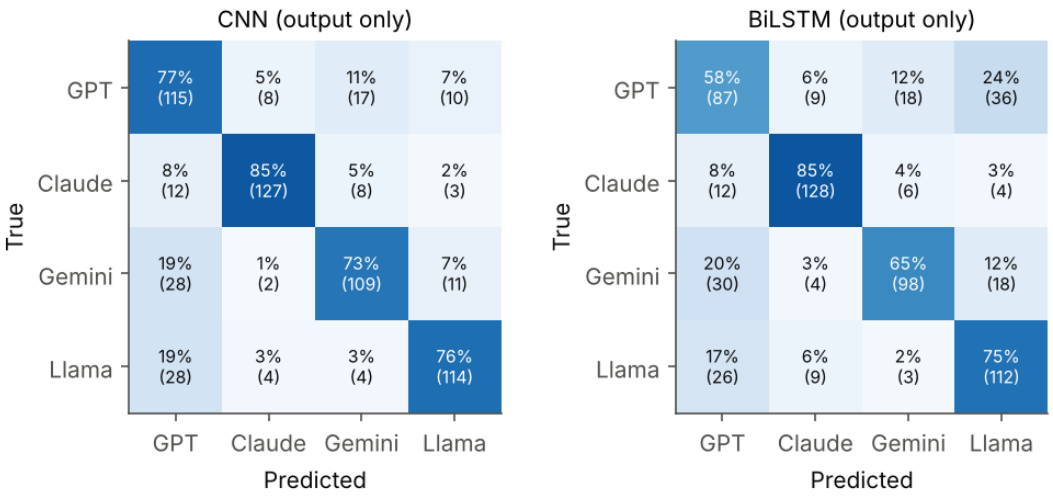


3.1 RQ1: Can we identify which LLM generated a response?

Yes. The response alone gave enough information to identify the model family much better than chance. The CNN reached 74.4 ± 2.9 macro-F1 and the BiLSTM reached 67.8 ± 2.2. Three results were especially interesting.

1. **CNN versus BiLSTM.** The CNN performed better even though the BiLSTM can use longer-range context. We think short writing patterns may have been easier to learn with only 2,800 training examples, but we did not run an experiment that proves this explanation.

2. **The simple baseline was strong.** TF-IDF with logistic regression reached 74.6 macro-F1, almost the same as the CNN's 74.4, and it trained in under a minute. This was an important comparison because a more complicated neural model was not automatically better for this dataset.
3. **Some families were easier to recognize.** CNN per-family F1 was 87.5 for Claude, 73.7 for Llama, 73.0 for Gemini, and 63.3 for GPT. The BiLSTM showed the same general pattern, with Claude highest at 82.5 and GPT lowest at 54.0. In the CNN confusion matrix, 68 of its 135 mistakes were predictions of GPT. One possibility is that GPT looked less consistent because the GPT category included eight versions, but the per-version test groups were small, so we cannot be certain.



3.2 RQ2: Does the user's prompt help?

(i) Prompt only. Using only the prompt worked slightly better than chance, with accuracy between 26.8% and 29.2% compared with 25% chance. In one-sided binomial tests on the averaged predictions of the three seeds, the TF-IDF result ($p = 0.011$) and the BiLSTM result ($p = 0.007$) were above chance, but the improvement was only a few percentage points, and the CNN result was not significant ($p = 0.33$).

(ii) Prompt plus response. Adding the prompt did not help. Compared with output only, macro-F1 dropped by 4.6 points for TF-IDF, 3.6 for the CNN, and 5.2 for the BiLSTM. We suspect that the prompt encoder added complexity without much useful model-family information, but we did not isolate that cause in a separate experiment.

(iii) Why prompt-only prediction might work well in other datasets. If different families are asked different types of questions, a classifier could learn the question type instead of anything about the model. We tested this possibility next.

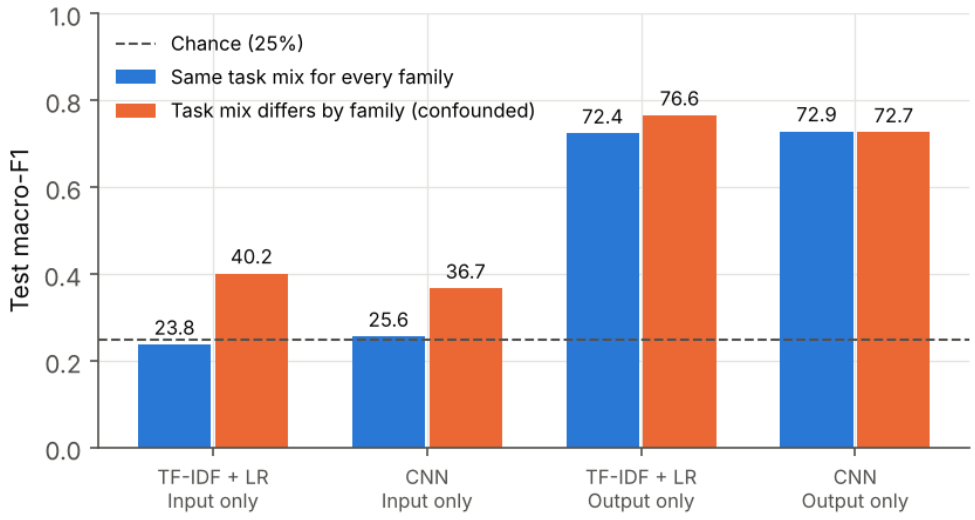
4. In-Depth Analyses

4.1 RQ2 (iii): when does input-only work “surprisingly well”?

To test this, we made two comparison datasets from 39,873 cleaned examples that were not used in the main sample, after excluding overlapping prompts. Both datasets had 2,184 training examples and about 478 test examples. In condition A, each family had about the same mix of task types. In condition B, we deliberately changed the mix so that GPT was 70% coding, Claude was 70% writing, Gemini was 70% general, and Llama was evenly mixed. This was a simulation of a biased data-collection process and not a claim about the real Arena dataset.

Model and view	A: same task mix (macro-F1)	B: different task mix (macro-F1)
TF-IDF, input only	23.8	40.2
CNN, input only	25.6 ± 2.5	36.7 ± 0.6
TF-IDF, output only	72.4	76.6
CNN, output only	72.9 ± 1.7	72.7 ± 1.2

With different task mixes, TF-IDF prompt-only macro-F1 rose from 23.8 to 40.2, and CNN prompt-only macro-F1 rose from 25.6 to 36.7. Output-only scores stayed high in both conditions. This shows why dataset construction matters, since a prompt-only classifier can appear to identify the model when it is really recognizing which kinds of prompts were sent to each family.



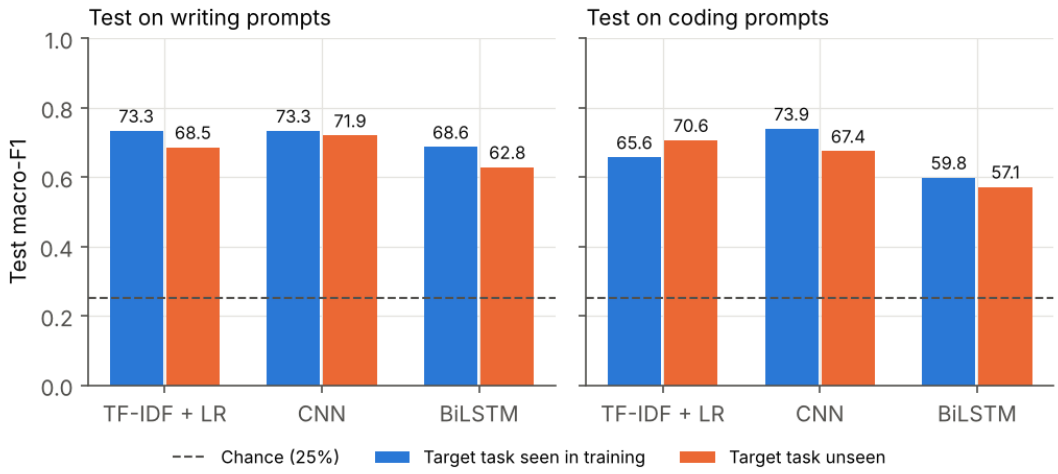
4.2 RQ3 (extra credit): do fingerprints generalize to unseen task types?

We tested whether the models could identify a family on a task type they had not seen in training. In one experiment, writing prompts were removed from training and the models were tested on writing. In the other, coding prompts were removed and the models were tested on coding. For a fair comparison, control models were trained on the same number of examples but included the target task. The training

sizes were 2,503 for writing and 2,451 for coding, and the held-out test groups had 136 writing and 152 coding examples. The held-out task was not used for training or early stopping.

Test task	Model	Target task seen	Target task unseen
Writing	TF-IDF + LR	73.3	68.5
Writing	CNN	73.3 ± 1.0	71.9 ± 4.1
Writing	RNN (BiLSTM)	68.6 ± 3.9	62.8 ± 2.1
Coding	TF-IDF + LR	65.6	70.6
Coding	CNN	73.9 ± 2.3	67.4 ± 4.2
Coding	RNN (BiLSTM)	59.8 ± 2.8	57.1 ± 4.7

Five of the six comparisons dropped when the task type was unseen, usually by a few macro-F1 points. The test sets were small (136 and 152 prompts), so these drops are not statistically significant, since every 95% confidence interval included zero. The CNN still scored 71.9 on unseen writing and 67.4 on unseen coding, both well above chance. Claude was relatively stable, while some per-family changes were much larger. For example, CNN Gemini recall on coding fell from 80.3% to 53.0%. Because those individual groups were small, we treat them cautiously. Overall, the results suggest that some writing patterns transfer between tasks, but we cannot say they are unaffected by task type.

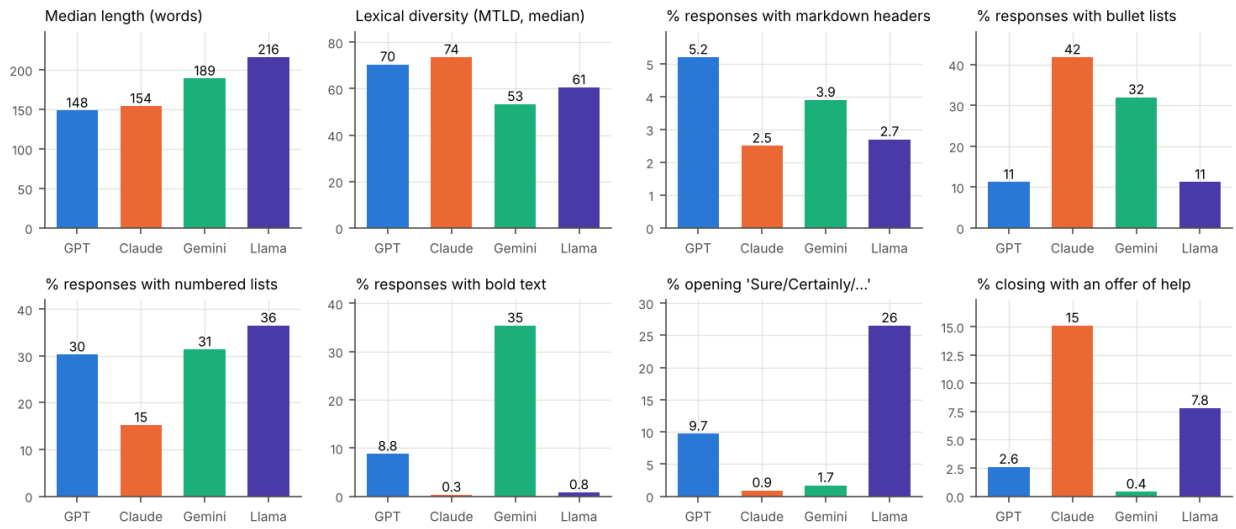


4.3 RQ4 (extra credit): what distinguishes the LLMs?

To understand what the classifiers might be noticing, we compared several kinds of writing patterns. We used feature counts, the strongest TF-IDF features, CNN saliency, and experiments where parts of the text were removed.

Style measurements. We calculated 27 features for each response, including length, lexical diversity, lists, bold text, and common opening phrases. All 27 differed across families in the Kruskal-Wallis tests ($p < 0.01$), although some differences were much larger than others. Bold formatting, “Sure/Certainly” openings, bullet lists, and sentence length had the largest effect sizes. The following image shows some

examples of what we found. A gradient-boosting model using all 27 features reached 67.5 macro-F1, compared with the CNN's 74.4, and no single feature group explained the whole result.



TF-IDF clues. We looked at the highest-weight words and short phrases from logistic regression. GPT had features such as "might," "various," "However," and "Remember." Claude had "Here is," "So in summary," and bullet markers. Gemini had bold markers (**), and Llama had "Sure," "Here's," and "help you." These are associations in this dataset and not rules that every response follows.

CNN clues. We also used gradient-times-input saliency to see which tokens were influential for the CNN. Many matched the TF-IDF patterns. GPT had words like "might" and "could," Claude had "Here" and "summary," Gemini had bold markers, and Llama had "Sure" and exclamation marks.

Taken together, these checks suggest that Gemini's bold formatting, Llama's frequent "Sure" openings, Claude's bullet lists and summaries, and GPT's more cautious wording all helped distinguish the families in this dataset.

Removing information (ablations). We removed different text signals from both the training and test data and then retrained the classifiers. We did this instead of only editing the test set so the models could adapt to the missing information. The table shows how much macro-F1 changed when we removed formatting, case and punctuation, everything after the first 50 words, or all three.

Signal removed	TF-IDF + LR	CNN	RNN (BiLSTM)
None (original text)	74.6	74.4	67.8
Formatting (markdown and line breaks)	−3.4	−10.0	−5.4
Case and punctuation	−8.1	−12.3	−4.6
Everything after the first 50 words	−6.2	−6.6	−0.8
All three	−13.1	−15.3	−8.4

The CNN was most affected by removing punctuation and formatting. Its macro-F1 fell by 12.3 points without case and punctuation and by 10.0 points without formatting. The BiLSTM lost only 0.8 points when limited to the first 50 words, perhaps because openings contain useful phrases, although we did not inspect its attention weights to confirm this. Even with all three changes, the models still scored roughly 59 to 62 macro-F1. Therefore, the signal is not just one obvious formatting trick, and word choice also matters.

4.4 Limitations

- The models are 2023 to January 2024 versions and not current ones, and each family mixes versions.
- Each family has only 1,000 examples and the test set has 600, so differences of one or two points between models are not meaningful.
- Gemini and Llama come from different periods of Arena collection, so part of the accuracy could come from era-specific style instead of the model itself.
- Task labels come from keyword rules, and the math label is only 50% precise.
- We used a single fixed split with three seeds and did not use cross-validation.

5. Lessons and Experience

- **Avoiding leakage.** We learned early that repeated Arena prompts could make a random train and test split misleading. Grouping by prompt took extra care, but it made the prompt-only comparison much more trustworthy.
- **Trying a baseline first.** TF-IDF with logistic regression reached 74.6 macro-F1, almost the same as the CNN. Without that comparison, we might have given the CNN too much credit just because it was a neural network.
- **Comparing model complexity.** The BiLSTM was more complicated to train but scored 67.8, below the CNN's 74.4. For this dataset, detecting short local patterns seemed more useful than learning long sequences.
- **Tuning matters.** The BiLSTM was still improving near our original 15-epoch limit, and some logistic-regression runs selected the largest C we allowed. We increased the epoch cap and widened the C grid instead of leaving those settings at arbitrary boundaries.
- **Checking repeatability.** CNN performance varied by seed, so we reported the mean and standard deviation instead of picking one good run.
- **Testing the pipeline.** Using synthetic examples with known patterns was useful before training on the real Arena data. It let us check whether the models and interpretation code could recover something we already knew should be there. If we extended this project, we would test newer LLM versions, use more data, and repeat the train and test split.

References

[1] L. Zheng et al., “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” NeurIPS Datasets and Benchmarks, 2023.

[2] LMSYS, lmsys-arena-human-preference-55k dataset. https://huggingface.co/datasets/lmsys/lmsys-arena-human-preference-55k

[3] Y. Kim, “Convolutional Neural Networks for Sentence Classification,” EMNLP, 2014.